Programa:	BurnInTest 4.0
Protección:	Empakado con Armadillo + Trial 30 días
Objetivos:	Desempakar y fulminar el Trial
URL descarga:	http://www.passmark.com/
Dificultad:	Súper fácil
Herramientas:	OllyDbg v1.10 + ImpRec 1.6 + PEiD/RDG
Cracker:	SyXe'05 ... [hAcK-c0d3d] ... [CracksLatinoS] Fecha: 28-10-05

Hola a todos los CracksLatinoS..!!

Hoy vamos a tratar de desempakar un Armadillo de los más sencillitos que he visto, sin duda le han aplicado el menor número de protecciones que se le pueden poner al .exe, de todos modos un Armadillo siempre nos va a dar problemas, eso sí la mayoría son viejos conocidos por todos, el programa víctima será el BurnItTest en su versión 4.0 recuerdo que la 3.0 no estaba protegida ni con Arma ni con nada, bueno como siempre abrimos el PEiD o el RDG Packer y miramos la pieza ;-)

Armadillo 3.00a - 3.61 -> Silicon Realms Toolworks

Vale un Arma 3.00 – 3.61 el RDG nos dice 3.01-3.05 con la detección tipo M-A y 3.75-4.x con la del tipo M-B ¿a cual creer? Pues no lo sé, he leído tutes en los que según dice el autor hay un momento en el que se puede pillar la versión exacta pero bueno, a nosotros esto ahora mismo no nos interesa demasiado, carguemos el ejecutable de la aplicación "bit.exe" en Olly:

```
00516000   60              PUSHAD
00516001   E8 00000000     CALL bit.00516006
00516006   5D              POP EBP
00516007   50              PUSH EAX
00516008   51              PUSH ECX
00516009 v EB 0F           JMP SHORT bit.0051601A
0051600B   B9 EB0FB8EB     MOV ECX,EBB80FEB
00516010   07              POP ES                    Modification of segment register
```

típico inicio en Armadillo, ese PUSHAD delator, jeje.. aquí no valen el método del PUSHAD-POPAD como en UPX y cosas así, quedando descartado de entrada, de momento pulsamos F9 y el programa corre bien bajo Olly (se deja debuggear bien) esto es bueno para nosotros, esto promete ;-)

Una vez lanzada la aplicación si ejecutamos LordPE y miramos entre los procesos cargados en memoria vemos que crea un hijo:

Path	PID	ImageBase	ImageSize
c:\cracking\tools\rdg packer\rdg packer det...	000001D0	00400000	00086040
c:\archivos de programa\burnintest 4.0\bit.exe	00000410	00400000	0020D000
c:\archivos de programa\burnintest 4.0\bit.exe	000007E0	00400000	0020D000
c:\cracking\tools\lordpe\lordpe.exe	00000488	00400000	00036000

Esto indica en un principio que es posible que nos encontremos Nanomites o bien que tenga CopyMemII (o ambas cosas posiblemente) bueno esto lo vamos a ver muy pronto lo que tiene, para salir de dudas rápido nos vamos a valer de nuestra gran amiga la api OpenMutexA, vamos a ello… Estando en el EP del packer (dirección de memoria :516000) vamos al CommandBar y tecleamos allí:

Command : bp OpenMutexA

Si no se queja el BP está puesto, perfecto! Pulsamos F9 para que corra el programa y Olly se detiene por primera vez:

```
77E62391   55              PUSH EBP
77E62392   8BEC            MOV EBP,ESP
77E62394   51              PUSH ECX
77E62395   51              PUSH ECX
77E62396   837D 10 00      CMP DWORD PTR SS:[EBP+10],0
```

Pero aún no haremos nada esperaremos a la segunda vez, o sea pulsamos F9 de nuevo y Olly vuelve a parar, quitamos el BP y retornamos de la api con Ctrl+F9, vemos esto:

```
004FC13D   85C0            TEST EAX,EAX
004FC13F   0F85 7A020000   JNZ bit.004FC3BF
004FC145   6A 01           PUSH 1
004FC147   FF15 30605200   CALL DWORD PTR DS:[<&KERNEL32.GetCurren  kernel32.GetCurrentThread
004FC14D   50              PUSH EAX
004FC14E   FF15 2C605200   CALL DWORD PTR DS:[<&KERNEL32.SetThread  kernel32.SetThreadPriority
004FC154   C685 54F9FFFF   MOV BYTE PTR SS:[EBP-6AC],0
004FC15B   68 38655200     PUSH bit.00526538                       ASCII "Kernel32"
004FC160   FF15 64615200   CALL DWORD PTR DS:[<&KERNEL32.LoadLibra  kernel32.LoadLibraryA
004FC166   8985 50F9FFFF   MOV DWORD PTR SS:[EBP-6B0],EAX
004FC16C   83BD 50F9FFFF   CMP DWORD PTR SS:[EBP-6B0],0
004FC173   74 32           JE SHORT bit.004FC1A7
004FC175   68 24655200     PUSH bit.00526524                       ASCII "IsDebuggerPresent"
004FC17A   8B85 50F9FFFF   MOV EAX,DWORD PTR SS:[EBP-6B0]
004FC180   50              PUSH EAX
004FC181   FF15 6C615200   CALL DWORD PTR DS:[<&KERNEL32.GetProcAd  kernel32.GetProcAddress
004FC187   8985 B8F8FFFF   MOV DWORD PTR SS:[EBP-748],EAX
004FC18D   83BD B8F8FFFF   CMP DWORD PTR SS:[EBP-748],0
004FC194   74 11           JE SHORT bit.004FC1A7
```

Esta zona sí nos interesa, debo decir que en casi todos los Arma que vi es la segunda llamada la que nos interesa pero una única vez era en la primera, entonces ¿cómo sabemos cual es la correcta? La respuesta la tenemos en la imagen anterior, sea la llamada que sea (la segunda, primera, tercera, cuarta…) siempre veremos las referencias a IsDebuggerPresent y a GetProcAddress, esto sin duda nos ratifica que es la llamada correcta y en la que debemos de actuar, superamos la api y vemos que quedó EAX==0, quiere decir que va a crear un hijo pero no le dejaremos, jeje…

```
Registers (FPU)
EAX 00000001
ECX 77F4119F ntdll.77F4119F
EDX 00000002
EBX 0050F8D9 bit.0050F8D9
ESP 0012F57C
EBP 0012FCD8
ESI 00000000
EDI 0012FEFC

EIP 004FC13D bit.004FC13D
```

modificamos EAX con tal de que quede con un valor distinto de 0 el que sea no importa ese valor sólo se utiliza para el salto luego en la llamada a GetCurrentThreadId se sobrescribirá el valor de EAX ya que traerá el valor de retorno de la api, modificamos EAX y ya no se crea el hijo de esta manera podemos debuggear al proceso ya que es único y no está siendo depurado (recordad que un proceso sólo puede ser depurado unaúnica vez, si el padre debuggea al hijo nosotros no tenemos acceso a él hasta que el padre lo libere) y con esto lo que conseguimos es que el padre no cree ningún hijo pensando que ya existe ;-)

```
004FC13D   85C0            TEST EAX,EAX
004FC13F   0F85 7A020000   JNZ bit.004FC3BF
004FC145   6A 01           PUSH 1
004FC147   FF15 30605200   CALL DWORD PTR DS:[<&KERNEL32.GetCurren  kernel32.GetCurrentThread
004FC14D   50              PUSH EAX
004FC14E   FF15 2C605200   CALL DWORD PTR DS:[<&KERNEL32.SetThread  kernel32.SetThreadPriority
004FC154   C685 54F9FFFF   MOV BYTE PTR SS:[EBP-6AC],0
004FC15B   68 38655200     PUSH bit.00526538                       ASCII "Kernel32"
004FC160   FF15 64615200   CALL DWORD PTR DS:[<&KERNEL32.LoadLibra  kernel32.LoadLibraryA
004FC166   8985 50F9FFFF   MOV DWORD PTR SS:[EBP-6B0],EAX
004FC16C   83BD 50F9FFFF   CMP DWORD PTR SS:[EBP-6B0],0
004FC173   74 32           JE SHORT bit.004FC1A7
004FC175   68 24655200     PUSH bit.00526524                       ASCII "IsDebuggerPresent"
```

Ahí vemos como salta y no crea el hijo, además evitamos la llamada a IsDebugger, jeje.. Esto está muy bien pero tiene sus desventajas sobretodo en exes protegidos con CopyMemII ya que no habrá padre que desencripte los bloques de código y no correrá, lo mismo ocurre con los Nanomites pero este es un tema aparte porque sí podemos obtener un dumpeado cosa que de la manera anterior no. Una vez hecho esto ponemos un BPM en la primera sección (la de código):

```
00260000 00001000                    Map  RW   RW
00270000 00016000                    Map  R    R    \De
00290000 00034000                    Map  R    R    \De
002D0000 00041000                    Map  R    R    \De
00320000 00006000                    Map  R    R    \De
00330000 00041000                    Map  R    R
00380000 00001000                    Priv RW   RW
00390000 00001000                    Priv RW   RW
003A0000 00002000                    Priv RW   RW
003B0000 00003000                    Map  R    R    \De
00400000 00001000 bit        PE header Imag R    RWE
00401000 00069000 bit  .text
0046A000 0000A000 bit  .rdat
00474000 00071000 bit  .data
004E5000 00001000 bit  .data
004E6000 00030000 bit  .text
00516000 00010000 bit  .adat
00526000 00020000 bit  .data
00546000 00070000 bit  .pdat
005B6000 00057000 bit  .rsrc
00610000 00006000
006D0000 00002000
006E0000 00103000
007F0000 000CE000
77C40000 00001000 GDI32
77C41000 0003B000 GDI32  .text
```

Actualize
Dump in CPU
Dump
Search Ctrl+B
Set break-on-access F2
Set memory breakpoint on access
Set memory breakpoint on write

Y damos RUN, en este caso el programa llega al OEP sin problemas:

```
0045D6D1   55              PUSH EBP
0045D6D2   8BEC            MOV EBP,ESP
0045D6D4   6A FF           PUSH -1
0045D6D6   68 B0114700     PUSH bit.004711B0
0045D6DB   68 30BD4500     PUSH bit.0045BD30
0045D6E0   64:A1 00000000  MOV EAX,DWORD PTR FS:[0]
0045D6E6   50              PUSH EAX
0045D6E7   64:8925 000000  MOV DWORD PTR FS:[0],ESP
0045D6EE   83EC 58         SUB ESP,58
0045D6F1   53              PUSH EBX
0045D6F2   56              PUSH ESI
0045D6F3   57              PUSH EDI
0045D6F4   8965 E8         MOV DWORD PTR SS:[EBP-18],ESP
0045D6F7   FF15 D4A14600   CALL DWORD PTR DS:[46A1D4]
0045D6FD   33D2            XOR EDX,EDX
0045D6FF   8AD4            MOV DL,AH
```

Esto indica claramente que no hay CopyMemII, lo podríamos haber sabido poniendo un BP en WriteProcessMemory y mirar si se llama para que escriba bloquecitos de 1000h bytes, si sólo para dos veces para escribir 2 bytes (en el EP del hijo) es que no hay nada, cuando para sucesivamente copiando 1000h bytes es que efectivamente tiene CopyMem, en este caso está claro que no hay nada de esto, está limpio ;-)

Podríamos aprovechar ahora que estamos aquí para volcar (dumpear) el archivo al disco peeeeeeeeeeeero el Arma nos quiere hacer la vida imposible y nos ha modificado el header del archivo para que no se pueda trabajar bien con él, podríamos copiar la cabecera del proceso original (del disco) y pegarla quedaría reparado pero también se puede hacer de un modo más artesanal que es el que a mi personalmente me gusta más, principalmente hay dos problemas en la cabecera, uno grave y otro no tanto:

- grave: el PE Signatura apunta a una zona incorrecta
- menos grave: el número de secciones no se corresponde con el real

de estos dos el primero de ellos es el primero que hay que tratar si queremos volcar bien el proceso, he de decir que ha habido veces que no he hecho nada y el dumpeado chutaba bien (aunque todavía no entiendo bien el porqué). Para reparar el primer error basta con ir a la dirección del PE Signatura que siempre está en :40003C y allí cambiar el valor que haya por el correcto, el valor correcto es el offset de la dirección donde se encuentra la cadena "PE" dentro de la cabecera, veamos la imagen:

```
Address   Hex dump                                         ASCII
00400000  4D 5A 90 00 03 00 00 00 04 00 00 00 FF FF 00 00  MZÉ.♥...♦...  ..
00400010  B8 00 00 00 00 00 00 00 40 00 00 00 00 00 00 00  ¸.......@.......
00400020  00 00 35 E9 0A 00 00 00 00 00 00 00 00 00 00 00  ..5ú............
00400030  00 00 00 00 00 00 00 00 00 00 00 00 B3 88 16 00  ............│ê▬.
00400040  0E 1F BA 0E 00 B4 09 CD 21 B8 01 4C CD 21 54 68  ♫▼│♫.┤.=↑@@L=↑Th
00400050  69 73 20 70 72 6F 67 72 61 6D 20 63 61 6E 6E 6F  is program canno
00400060  74 20 62 65 20 72 75 6E 20 69 6E 20 44 4F 53 20  t be run in DOS
00400070  6D 6F 64 65 2E 0D 0D 0A 24 00 00 00 00 00 00 00  mode....$.......
00400080  D1 78 F2 35 95 19 9C 66 95 19 9C 66 95 19 9C 66  Ðx=5ò↓£fò↓£fò↓£f
00400090  EE 05 90 66 96 19 9C 66 FA 06 97 66 99 19 9C 66  ¯♣£fû↓£f·♠ûf ↓£f
004000A0  16 05 92 66 B2 19 9C 66 FA 06 96 66 0F 19 9C 66  ▬♣Æf▒↓£f·♠ûf*↓£f
004000B0  95 19 9C 66 9D 19 9C 66 69 39 8E 66 94 19 9C 66  ò↓£f¥↓£fi9Äfö↓£f
004000C0  4F 3A 81 66 C6 19 9C 66 4F 3A 80 66 90 19 9C 66  O:üfã↓£fO:ÇfÉ↓£f
004000D0  A6 3B B9 66 97 19 9C 66 6F 3A 85 66 80 19 9C 66  ª;¦fù↓£fo:àfÇ↓£f
004000E0  95 19 9D 66 FC 18 9C 66 C1 3A AD 66 DF 19 9C 66  ò↓Øf↑↑£f┴:¡f▀↓£f
004000F0  52 1F 9A 66 94 19 9C 66 52 69 63 68 95 19 9C 66  R▼üfö↓£fRichò↓£f
00400100  00 00 00 00 00 00 00 00 50 45 00 00 4C 01 0C 00  ........PE..L♪..
00400110  B5 B0 C4 42 00 00 00 00 00 00 00 00 E0 00 0F 01  µ└─B........à.☼☺
00400120  0B 01 53 52 00 C0 03 00 00 A0 0C 00 00 00 00 00  ♂☺SR.└♥..á.....
```

lo que está en recuadro rojo es el PE Signatura, como dije siempre está en :40003C por Standard y allí vemos el valor 1688B3 que sumado el imagebase quedaría 5688B3, si vamos a esa dirección vemos:

```
Address   Hex dump                                         ASCII
005688B3  50 45 00 00 4C 01 09 00 B5 B0 C4 42 00 00 00 00  PE..L♪○.µ└─B....
005688C3  00 00 00 00 E0 00 0F 01 0B 01 53 52 00 C0 03 00  ....à.☼☺♂☺SR.└♥.
005688D3  00 A0 0C 00 00 00 00 00 60 11 00 00 60 0E 00 00  .á.......`◄..`♫.
005688E3  00 60 12 00 00 00 40 00 00 10 00 00 00 10 00 00  .`↕...@..►...►..
005688F3  04 00 00 00 00 00 00 00 04 00 00 00 00 00 00 00  ♦.......♦.......
```

Efectivamente allí está la cadena "PE" pero esta no es la cabecera del archivo lógicamente está al inicio del mismo :400000-400FFF es por esto que nos da error, lo que tenemos que hacer es buscar la cadena "PE" pero dentro de la cabecera, en una imagen anterior la encontramos en :400108 si le restamos el imagebase quedará 108h pues este valor es el que debe contener el PE Signatura y no el 1688B3 que tiene:

```
Address   Hex dump                                         ASCII
00400000  4D 5A 90 00 03 00 00 00 04 00 00 00 FF FF 00 00  MZÉ.♥...♦...  ..
00400010  B8 00 00 00 00 00 00 00 40 00 00 00 00 00 00 00  ¸.......@.......
00400020  00 00 35 E9 0A 00 00 00 00 00 00 00 00 00 00 00  ..5ú............
00400030  00 00 00 00 00 00 00 00 00 00 00 00 08 01 00 00  ............◘☺..
00400040  0E 1F BA 0E 00 B4 09 CD 21 B8 01 4C CD 21 54 68  ♫▼│♫.┤.=↑@@L=↑Th
00400050  69 73 20 70 72 6F 67 72 61 6D 20 63 61 6E 6E 6F  is program canno
```

Vamos allí y lo cambiamos, de momento si vamos a la ventanita de memoria (Alt+M) veremos que la primera sección vuelve a tener el distintivo "PE header" quehabía perdido, esto es señal de que vamos muy bien ;-)

El otro tema no tan grave pero sí molesto es el tema de las secciones, si miramos la primera sección del archivo (la cabecera) desde Olly en modo "Special View" y vamos bajando con la rueda del ratón veremos las secciones con sus respectivos valores, llegará un momento en que estos valores serán todo ceros pues todas las que aparezcan así son sobrantes y no deberían estar, si por ejemplo el archivo dice tener 15 secciones y miramos vemos 3 que están llenas de ceros en realidad tendrá 12 secciones, pues modificaríamos el 15 por el 12 y con esto ya podemos ahora sí dumpear ;-DD

Yo utilizo Ollydump que me va perfecto, dumpeamos como siempre destildando la opción "Rebuild Imports" que no funciona muy bien, lo llamamos dump.exe por ejemplo. Ya tenemos el dumpeado pero todavía no corre porque no tiene tabla de importaciones, vamos a repararla que también está dañada Armadillo nos redirigió parte de la misma…

Recuperación de la IAT…

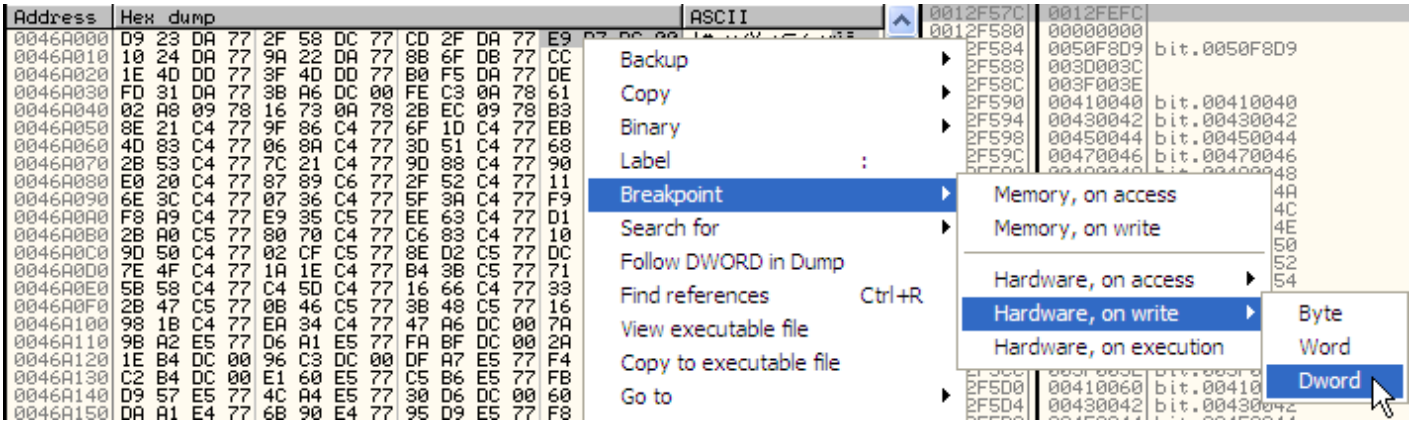
Reiniciamos Olly pero antes ns fijamos en algún puntero inválido como este por ejemplo:

```
Address   Hex dump                                          ASCII
0046A000  D9 23 DA 77 2F 58 DC 77 CD 2F DA 77 E9 D7 DC 00
0046A010  10 24 DA 77 9A 22 DA 77 8B 6F DB 77 CC F7 DA 77
0046A020  1E 4D DD 77 3F 4D DD 77 B0 F5 DA 77 DE EB DA 77
0046A030  FD 31 DA 77 3B A6 DC 00 FE C3 0A 78 61 6D 0A 78
```

le echamos la foto y nos quedamos con la dirección de la entrada y su valor final:

[46A00C]==DCD7E9

Esto nos servirá para localizar el momento exacto en que redirecciona la IAT, sólo debemos de poner un 'HBP On Write' en dicha entrada:



lo debemos de poner Hardware sino no para, ahora reiniciamos Olly y evitamos que se cree un segundo proceso con ayuda de la api OpenMutexA como vimos antes, damos RUN y Olly para, no se han escrito los valores finales de la entrada así que F9 nuevamente:

```
00D148D2  73 1D          JNB SHORT 00D148F1
00D148D4  8B85 1CE8FFFF  MOV EAX,DWORD PTR SS:[EBP-17E4]
00D148DA  8B8D 80E2FFFF  MOV ECX,DWORD PTR SS:[EBP-1D80]
00D148E0  8908           MOV DWORD PTR DS:[EAX],ECX        ;Aqui guarda...
00D148E2  8B85 1CE8FFFF  MOV EAX,DWORD PTR SS:[EBP-17E4]   bit.0046A00C
00D148E8  83C0 04        ADD EAX,4
00D148EB  8985 1CE8FFFF  MOV DWORD PTR SS:[EBP-17E4],EAX
00D148F1  E9 36FDFFFF    JMP 00D1462C
```

En esta instrucción rellena la IAT de valores buenos y de valores malos, ECX le trae estos valores, debemos tracear el loop en que nos encontramos en busca del salto mágico, una buena estrategia se basa en buscar el CALL a GetProcAddress y que pone en EAX la api correcta y una vez localizado mirar por encima a ver si vemos algún salto que de producirse supere la dirección del CALL y no pase, entonces de esta manera deja en EAX un valor malo, con este método se localiza el mágico en un momento, hay casos en que no es así ya que el mágico está dentro del CALL que pone en EAX la api, en esos casos se entra en el CALL y se sigue mirando. Para este caso concreto no necesitaremos entrar en ningún CALL ya que la respuesta la tenemos cerca del CALL, una vez localizado miramos por encima para encontrar el salto buscado (el magico):

```
00D147D9   83BD 80E2FFFF  CMP DWORD PTR SS:[EBP-1D80],0
00D147E0 v 75 3F          JNZ SHORT 00D14821                    ;ojo a este salto..!!
00D147E2   0FB785 84E2FFF MOVZX EAX,WORD PTR SS:[EBP-1D7C]
00D147E9   85C0           TEST EAX,EAX
00D147EB v 74 0F          JE SHORT 00D147FC
00D147ED   0FB785 84E2FFF MOVZX EAX,WORD PTR SS:[EBP-1D7C]
00D147F4   8985 C4CDFFFF  MOV DWORD PTR SS:[EBP-323C],EAX
00D147FA v EB 0C          JMP SHORT 00D14808
00D147FC   8B85 7CE2FFFF  MOV EAX,DWORD PTR SS:[EBP-1D84]
00D14802   8985 C4CDFFFF  MOV DWORD PTR SS:[EBP-323C],EAX
00D14808   FFB5 C4CDFFFF  PUSH DWORD PTR SS:[EBP-323C]
00D1480E   FFB5 B8E4FFFF  PUSH DWORD PTR SS:[EBP-1B48]
00D14814   E8 CA53FEFF    CALL 00CF9BE3                         ;call GetProcAddress...
00D14819   59             POP ECX
00D1481A   59             POP ECX
00D1481B   8985 80E2FFFF  MOV DWORD PTR SS:[EBP-1D80],EAX
00D14821 > 83BD 80E2FFFF  CMP DWORD PTR SS:[EBP-1D80],0
00D14828 v 0F85 98000000  JNZ 00D148C6
00D1482E   0FB785 84E2FFF MOVZX EAX,WORD PTR SS:[EBP-1D7C]
00D14835   85C0           TEST EAX,EAX
00D14837 v 74 54          JE SHORT 00D1488D
00D14839   FF15 D8B0D100  CALL DWORD PTR DS:[D1B0D8]            ntdll.RtlGetLastWin32Error
00D1483F   83F8 32        CMP EAX,32
00D14842 v 75 0C          JNZ SHORT 00D14850
```

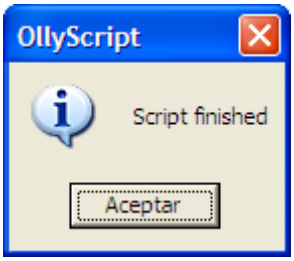
Ya lo tenemos, jeje… no fue difícil ya os lo decía, cada vez que se produce el salto guarda una entrada mala y cuando no se produce la guarda buena, está claro que es el mágico y que no debe saltar nunca. Podemos intentar nopearlo pero es casi seguro que no lleagremos al OEP, lo mejor será utilizar un mini_script que nos haga este trabajito:

```
-----------------/ inicio_script
start:
eob break
run

break:
cmp eip, 0DE47E0
jne exit
mov eip, 0DE47E2
jmp start

exit:
ret
-----------------/ final_script
```

lo único que hace es mirar si estamos en el mágico, si es así modificar el EIP para que apunte a la siguiente instrucción (simula el nopeo) y si no estamos en el mágico hemos terminado, antes de ejecutar el script debemos poner un HBP en la posición del mágico :DA47E0 para que pare y el script se ejecute bien, cuando termine veremos esto:



Ya terminó, si todo ha ido bien estaremos en el OEP con la tabla reparada, abrimos el ImpRec y le damos los valores correctos para trabajar:

OEP: 0005D6D1
RVA: 0006A000
Size: 0000052C

Damos a "Get imports" y nos aparecen 11 punteros inválidos, es muy probable que debamos reparar la cabecera para que el Import no se queje sino no reconoce bien al archivo, estos punteros son auténtica basura, en realidad son la separación entre dll's distintas y que deberían ser ceros, Armadillo nos coloca valores muy similares a los que pone en los punteros redirigidos para hacernos dudar pero es tontería ya que se ve fácil que son las separaciones, entonces "Cut Thunks" y "Fix dump" elegimos dump.exe y nos creará dump_.exe, si ahora probamos este último archivo y de momento entramos sin ver la nag ¿? Lo estraño de todo es que ésta es propia del programa y no de Armadillo, no me explico porqué no aparece, bueno cosas que pasan ;-)

Lo mejor de todo es que tras desempakr el archivo se comporta como si estuviera en la versión registrada, no vemos nags, no expira, etc., para colmo veamos en el About:

jeje… Registered Version pone bien clarito, si miramos el empakado aparte de mostrarnos la nag aquí en el About pone Unregistered User.

Esto quiere decir que el programa está parcheado, espero que os hayáis divertido, para cualquier cosa me podéis encontrar como siempre en:

syxe00@yahoo.es